

Università degli studi di Salerno



Progetto di Natural Language Processing

INTERROGAZIONE DI CONTENUTI VIDEO TRAMITE SISTEMA RAG

Malafrente Sabato – Di Lauro Antonio

Prof. Loredana caruccio - dott. Eleonora Calò

Anno Accademico 2025/2026

INDICE

1. Introduzione

1.1 Strumenti utilizzati

2. Progetto in python

2.1 Configurazioni iniziali

2.2 Gestione dei video youtube

2.3 Gestione dei chunk dinamici

2.4 Metriche di valutazione

2.5 Gestione delle routes

2.6 Interfaccia html

3. Conclusioni

1. INTRODUZIONE

L'obiettivo di questo progetto è quello di creare un sistema RAG (Retrieval-augmented generation) per recuperare informazioni in video YouTube inseriti tramite interfaccia utente, utilizzando opportune tecniche di NLP, retrieval dinamico di chunks dal database vettoriale e interazione con LLM scaricati da LM Studio, valutando le risposte ai prompt con metriche opportune dalla libreria ragas.

1.1 Strumenti utilizzati

Per lo svolgimento del progetto sono stati utilizzati diversi strumenti software:

- Flask: framework web utilizzato per gestire la connessione tra l'interfaccia html e il modulo di elaborazione delle richieste;
- NLTK: libreria per i task di Natural Language Processing;
- Punkt e Punkt_tab: moduli che dividono il testo in frasi utilizzando un algoritmo non supervisionato. Ci sono serviti per effettuare la tokenizzazione delle frasi (sentence tokenization). Questi moduli sono in grado di riconoscere abbreviazioni, numeri decimali o punti di sospensione, evitando che le frasi si interrompano nel punto sbagliato;
- youtube-transcript-api: libreria che ci consente di recuperare i sottotitoli da un video youtube;
- intfloat/multilingual-e5-base: modello di embedding utilizzato per trasformare i chunks in vettori di embeddings, analizzando il contesto e comprimendolo in un vettore denso a 768 dimensioni;
- FAISS: database vettoriale di META interamente posto in memoria RAM, per conservare e recuperare i vettori di embeddings generati e per calcolarne, attraverso la similarità del coseno, la similarità semantica rispetto alla query posta dall'utente;
- OpenAI: libreria per configurare un client locale su <http://localhost:1234/v1>;
- LM Studio: software per eseguire modelli di intelligenza artificiale generativa in locale. Gli LLM vengono utilizzati nello specifico per porre domande sul contenuto del video YouTube. Quelli utilizzati nel progetto sono google/gemma-2-9b-it, qwen2.5-1.5b-instruct e meta-llama-3.1-8b-instruct.

2. Codice in Python

2.1 Configurazioni iniziali

Iniziamo preparando tutto ciò che ci servirà successivamente:

```
# SCARICHIAMO IL MODELLO PUNKT PER TOKENIZATION
try:
    nltk.data.find('tokenizers/punkt')
except LookupError:
    nltk.download('punkt')

try:
    nltk.data.find('tokenizers/punkt_tab')
except LookupError:
    nltk.download('punkt_tab')
```

Qui verifichiamo se i componenti di NLTK per la tokenizzazione (punkt e punkt_tab) sono già presenti, altrimenti li scarichiamo.

```
# CONFIGURIAMO IL CLIENT LOCALE
app = Flask(__name__)

client = OpenAI(base_url="http://localhost:1234/v1", api_key="lm-studio")

# MODEL_NAME = "qwen2.5-1.5b-instruct"
MODEL_NAME = "meta-llama-3.1-8b-instruct"
# MODEL_NAME = "gemma-2-9b-it"
```

Dopo aver inizializzato il framework Flask, inizializziamo il client OpenAI in modo che non punti ai server ufficiali di ChatGPT, ma che reindirizza il traffico in locale, attraverso la porta 1234 di localhost. Verrà quindi utilizzato con LM Studio che creerà un server locale che imiterà il funzionamento di OpenAI. Salviamo poi in una variabile il nome dell'LLM, che varierà nelle varie esecuzioni poiché sono tre gli LLM che utilizziamo e di cui attraverso le metriche si valuteranno le differenze. Il nome indicherà a LM Studio quale degli LLM scaricati in locale deve utilizzare per le risposte.

```
# CONFIGURAZIONE PARAMETRI CHUNKS DINAMICI

#CREAZIONE DEI CHUNK
CHUNK_SIMILARITY_THRESHOLD = 0.78
MAX_SENTENCES_PER_CHUNK = 12

#RETRIEVAL DEI CHUNK
RETRIEVAL_MAX_K = 6
RETRIEVAL_DISTANCE_THRESHOLD = 0.415
```

Una delle parti importanti del codice che vedremo in seguito è quella di dinamicizzare i chunks. Questo vuol dire in primis rendere dinamico il numero di frasi che compongono ogni chunk, con l'obiettivo di costruire chunks con frasi che trattano dello stesso topic così che chunks con topic diversi avranno una certa distanza di similarità e siano dunque facilmente distinguibili dall'algoritmo all'atto del retrieval. In secondo luogo, all'atto della query dell'utente bisogna selezionare dal db vettoriale i chunks pertinenti per fornire la risposta. Il numero di chunks recuperati varierà in base alla domanda, questo per evitare di recuperare chunks irrilevanti o di non recuperare chunks che invece erano utili per la risposta. Sia nel primo task che nel secondo, la dinamicità del numero di frasi per chunk e del numero di chunks da recuperare avverrà in base a dei parametri di threshold riportati nel codice sopra, stabiliti in base a regole che vedremo in seguito. Infine stabiliamo un limite massimo di frasi che possono essere inserite per chunk, per evitare di creare un chunk troppo grande da mandare in errore l'LLM se ci troviamo nella situazione in cui si parla di uno stesso concetto per tutto il video o comunque per periodi prolungati.

```
# SYSTEM PROMPT PER CHAT STANDARD
SYSTEM_PROMPT = """Sei un assistente universitario rigoroso e preciso.
Il tuo compito è rispondere alle domande dell'utente basandoti ESCLUSIVAMENTE sul contesto fornito.

REGOLE FONDAMENTALI:
1. Usa solo ed esclusivamente le informazioni presenti nel testo del contesto.
2. Se il contesto non contiene le informazioni necessarie per rispondere, devi scrivere esattamente e solo la frase: "Non lo so."
3. Non ripetere mai il testo del contesto o il prompt nella tua risposta. Rispondi direttamente alla domanda.
4. Non inventare dati e non usare le tue conoscenze pre-addestrate.
5. Rispondi sempre in lingua italiana, in modo chiaro, dettagliato ma conciso.
"""

# SYSTEM PROMPT NEL CASO L'UTENTE SCELGA LA MODALITA' CHAT SOCRATICA
SOCRATIC_PROMPT = """
Sei un docente universitario che usa rigorosamente il METODO SOCRATICO.
Il tuo obiettivo è guidare lo studente a comprendere il concetto da solo, senza MAI dare la risposta diretta o spiegazioni preconfezionate.
Devi rispondere utilizzando esclusivamente
le informazioni presenti nel contesto.
Regole:
- Non usare conoscenze esterne.
- Non inventare informazioni.
- Se la risposta non è contenuta nel contesto scrivi:
Non lo so.
Rispondi sempre in italiano.
Fornisci spiegazioni dettagliate ma concise.
"""
```

L'interfaccia html prevede che dopo il caricamento del video YouTube l'utente possa scegliere se interagire con l'LLM in modalità standard o in modalità socratica. In entrambi i casi abbiamo bisogno di istruire l'LLM: nel caso della modalità standard l'LLM deve rispondere solamente basandosi sulle informazioni della trascrizione del video, dunque non deve utilizzare le sue conoscenze derivate dall'addestramento e se una certa risposta non è presente nel video deve rispondere di non sapere. Nella modalità chat socratica invece, l'LLM oltre che rispettare le regole menzionate pocanzi, deve rispondere seguendo il metodo socratico, che consiste non nel dare una risposta in modo diretto, ma di guidare l'interlocutore ad arrivarci in maniera autonoma. A questo scopo sono stati stabiliti i due prompt in immagine, uno per l'utente che sceglie la modalità standard e l'altro se esso sceglie la modalità socratica, da inviare all'LLM insieme alla query.

```
embedding_model = SentenceTransformer("intfloat/multilingual-e5-base")

def cosine_similarity(vec1, vec2):
    return np.dot(vec1, vec2) / (np.linalg.norm(vec1) * np.linalg.norm(vec2))
```

Definiamo poi il modello di embedding che ci servirà in seguito, con il compito di generare un vettore di embeddings per ogni chunk. Definiamo poi la funzione di similarità del coseno che ci serve per calcolare la distanza tra embeddings. Si scrive dunque a codice la formula della similarità del coseno:

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}},$$

corrispondente al prodotto scalare dei vettori diviso il prodotto delle loro norme.

2.2 Gestione dei video youtube

Dopo aver effettuato le dovute configurazioni iniziali passiamo alla fase operativa: dopo che l'utente copia e incolla il link del video YouTube dall'interfaccia, come prima cosa recuperiamo il titolo del video, che ci servirà in seguito per tenere traccia dei vari video inseriti per mostrarli all'utente nell'interfaccia, in modo che non bisogna inserirli di nuovo:

```
def get_video_title(video_url):
    try:
        url = f"https://www.youtube.com/oembed?url={video_url}&format=json"
        headers = {'User-Agent': 'Mozilla/5.0'}
        req = urllib.request.Request(url, headers=headers)
        with urllib.request.urlopen(req) as response:
            data = json.loads(response.read().decode())
            return data.get("title", "Video senza titolo")
    except Exception as e:
        print(f"Impossibile recuperare il titolo: {e}")
        return "Video YouTube (Titolo non recuperabile)"
```

In particolare, attraverso il formato oEmbed si chiede al provider (YouTube) le informazioni su un link, restituendo un oggetto JSON contenente le varie informazioni del video tra cui il titolo. Viene impostato un user agent Mozilla per evitare che YouTube blocchi una tale richiesta credendo che si tratti di scraping di bot. Si apre poi effettivamente la connessione verso i server di YouTube e si scaricano i dati ricevuti convertendoli in una stringa di testo leggibile, trasformando poi la stessa in un dizionario Python. Infine viene estratto il valore associato alla chiave "title", cioè il titolo del video che stavamo cercando.

```

# ESTRAZIONE ID DEL VIDEO
def get_video_id_from_url(youtube_url):
    query = urlparse(youtube_url)

    if query.hostname in ['www.youtube.com', 'youtube.com']:

        if 'v' in query.query:
            return parse_qs(query.query)['v'][0]

        path = query.path.split('/')
        return path[-1]

    elif query.hostname == 'youtu.be':

        return query.path[1:]

    return None

```

A questo punto bisogna estrarre l'id univoco del video (una stringa alfanumerica di 11 caratteri che identifica la risorsa su YouTube). Questo passaggio è importante poiché altrimenti la libreria youtube-transcript-api non saprebbe quale video scaricare. Per prima cosa con la funzione `urlparse()` andiamo a scomporre l'URL del video in componenti distinte. Distinguiamo adesso due casi, quello di un link di YouTube standard, in cui compare la stringa `"www.youtube.com"` oppure `"youtube.com"`, e quello di un link condiviso, in cui compare invece la stringa `"youtu.be"`. Nel caso di un link YouTube standard è possibile trovarci in due casi, quello in cui nel link è presente il parametro `"?v="` e quello in cui questo non c'è: nel primo caso andiamo dunque a verificare se c'è la lettera `v`, e quando la trova utilizza la funzione `parse_qs()` per creare un dizionario python, per esempio `{'v': ['dQw4w9WgXcQ']}`, per poi estrarre il primo valore, corrispondente all'id del video. Nel secondo caso, che corrisponde a video Shorts o Embed, per recuperare l'id bisogna andare a ottenere la stringa dopo l'ultimo slash `"/"`. Nel caso invece da link video derivanti da una condivisione, dove il link è accorciato, l'id del video si trova immediatamente dopo il primo slash, dunque escludendo l'host (`youtu.be`), basta recuperare tutta la stringa escludendo il primo carattere (`"/"`).

```
# ESTRAZIONE TRASCRIZIONE DEL VIDEO
def get_video_transcript(video_url):
    video_id = get_video_id_from_url(video_url)
    yt_api = YouTubeTranscriptApi()

    try:
        transcript = yt_api.fetch(video_id, languages=['it'])
        print(f"Sottotitoli in italiano recuperati per il video: {video_id}")
    except Exception:
        try:
            transcript_list = yt_api.list_transcripts(video_id)
            transcript = transcript_list.find_transcript(['it', 'en']).fetch()
            print(f"Sottotitoli (Fallback Italiano/Inglese) recuperati per il video: {video_id}")
        except Exception as e:
            raise ValueError(
                f"Impossibile recuperare i sottotitoli per questo video. Verifica che siano abilitati su YouTube. Errore: {str(e)}")

    text = " ".join([row.text for row in transcript])
    return text
```

Il prossimo passo è quello di recuperare la trascrizione del video. Dopo aver ottenuto quindi l'id del video, inizializziamo il client della libreria youtube-transcript-api e invochiamo il metodo `fetch()`, il quale invia una richiesta ai server di YouTube per farci restituire i sottotitoli in lingua italiana o inglese del video in questione. A questo punto YouTube non restituisce il testo pulito dall'inizio alla fine, ma restituisce un dizionario in JSON in cui per ogni riga di sottotitolo compare il testo, il minuto di inizio e la durata, ad esempio:

```
{"text": "Benvenuti a questa lezione", "start": 0.12, "duration": 2.0}
```

Il nostro codice dunque estrae solo il campo `text` da ogni riga di sottotitolo, separandolo con uno spazio, ottenendo così un testo unico della trascrizione dell'intero video.

2.3 Gestione dei chunk dinamici

Arriviamo adesso alla parte riguardante la creazione dei chunks, il loro inserimento nel database vettoriale e il retrieval degli stessi, iniziando a vedere la parte relativa alla loro creazione. Creare dinamicamente i chunks significa scorrere tutte le frasi della trascrizione e scegliere di volta in volta se quella frase deve rimanere nel chunk corrente oppure crearne uno nuovo poiché è cambiato il contesto. Vediamo nello specifico:

```

# CREAZIONE DEI CHUNKS (DINAMICA E SEMANTICA)
def create_chunks(text, similarity_threshold=CHUNK_SIMILARITY_THRESHOLD, max_sentences=MAX_SENTENCES_PER_CHUNK)
    sentences = sent_tokenize(text, language='italian')

    if not sentences:
        return []
    print(f"Totale frasi da analizzare: {len(sentences)}")

    sentence_embeddings = embedding_model.encode(sentences, convert_to_numpy=True)

    chunks = []
    current_chunk = [sentences[0]]

    for i in range(1, len(sentences)):
        sim = cosine_similarity(sentence_embeddings[i], sentence_embeddings[i - 1])

        # Se l'argomento è lo stesso e non abbiamo superato il limite massimo di frasi
        if sim >= similarity_threshold and len(current_chunk) < max_sentences:
            current_chunk.append(sentences[i])
        else:
            # Tagliamo se cambia argomento oppure se il chunk è diventato troppo lungo
            if len(current_chunk) >= max_sentences:
                print(f"-> Limite di {max_sentences} frasi raggiunto.")
            else:
                print(f"-> Cambio argomento rilevato. Chiudo il chunk.")

            chunks.append(" ".join(current_chunk))
            current_chunk = [sentences[i]]

    if current_chunk:
        chunks.append(" ".join(current_chunk))
    print(f"Totale chunk dinamici creati: {len(chunks)}\n")
    return chunks

```

Per prima cosa utilizziamo `sent_tokenize()` dei modelli punkt di NLTK per effettuare sentence tokenization, cioè prendere il testo completo della trascrizione e trasformarlo in una lista di frasi separate. Con la funzione `embedding_model.encode()` il modello multilingual-e5-base crea un vettore numerico (embedding) per ciascuna frase dei sottotitoli. Viene inizializzata una lista vuota “chunks” inserendo al suo interno la prima frase, e di volta in volta con un ciclo for, per ogni frase si calcola la similarità del coseno con la frase precedente. Se non abbiamo superato il numero di frasi massimo consentite per chunk e se la similarità ottenuta è maggiore di un threshold (cioè il contesto rimane lo stesso della frase precedente), la frase corrente viene aggiunta al chunk corrente, altrimenti il chunk corrente viene chiuso e ne viene aperto un altro (cambio di argomento o nel caso venga raggiunto il limite massimo di frasi per un chunk).

Il valore di threshold per valutare la similarità tra due frasi consecutive è stato scelto essere 0.78. Questa decisione è stata presa sulla base del risultato di

alcuni test effettuati su vari video per trovare il valore ottimale. In particolare sappiamo che il numero medio di frasi per chunk in un sistema RAG è generalmente compreso in un range di 6-10 frasi [1], anche nel caso di trascrizioni da video Youtube [2]. A questo proposito abbiamo effettuato dei test, con i risultati riportati nelle tabelle di seguito, su cinque trascrizioni di video YouTube che trattano temi differenti (telegiornale, documentario, spettacolo televisivo, lezione universitaria, highlights di una partita di calcio), dove per ognuno di essi abbiamo creato una tabella in cui sulle righe facciamo variare i diversi valori di threshold, e sulle colonne riportiamo il numero di chunks creati dall'algoritmo e il valore Frasi/Chunk, cioè il numero medio di frasi per chunk, il valore che ci interessa mantenere nel range 6-10. Le tabelle sono riportate di seguito:

TELEGIORNALE: <https://www.youtube.com/watch?v=vNOHdve4CIE>

Soglia (Threshold)	Numero di Chunk	Media Frasi/Chunk
0.75	6	25.8
0.76	9	17.2
0.77	13	11.9
0.78	17	9.1
0.79	28	5.5
0.80	49	3.2

DOCUMENTARIO: <https://www.youtube.com/watch?v=EtJb5fafOkc>

Statistiche basate su un totale di 666 frasi:

Soglia (Threshold)	Numero di Chunk	Media Frasi/Chunk	
0.75	8	83.2	
0.76	12	55.5	
0.77	19	35.1	
0.78	56	11.9	
0.79	103	6.5	
0.80	166	4.0	

SPETTACOLO TELEVISIVO: <https://www.youtube.com/watch?v=qonQz-l87aQ>

Statistiche basate su un totale di 501 frasi:

Soglia (Threshold)	Numero di Chunk	Media Frasi/Chunk	
0.75	9	55.7	
0.76	21	23.9	
0.77	42	11.9	
0.78	73	6.9	
0.79	108	4.6	
0.80	170	2.9	

LEZIONE UNIVERSITARIA: <https://www.youtube.com/watch?v=xZPYc3CU1Do>

Statistiche basate su un totale di 530 frasi:

Soglia (Threshold)	Numero di Chunk	Media Frasi/Chunk
0.75	7	75.7
0.76	15	35.3
0.77	25	21.2
0.78	50	10.6
0.79	82	6.5
0.80	116	4.6

HIGHLIGHTS PARTITA DI CALCIO:

<https://www.youtube.com/watch?v=lFNneepo5M>

Statistiche basate su un totale di 63 frasi:

Soglia (Threshold)	Numero di Chunk	Media Frasi/Chunk
0.75	1	63.0
0.76	2	31.5
0.77	2	31.5
0.78	5	12.6
0.79	8	7.9
0.80	13	4.8

Dai risultati ottenuti, due valori di threshold rispettavano la caratteristica dell'avere la media di tutti e 5 i video nel range di 6-10 frasi per chunk, abbiamo dunque scelto il valore con più frasi/chunk dei due per evitare di avere avere più chunk troppo corti, e questo è 0.78, con una media di 10 frasi per chunk. Dunque alla fine avremo che se la similarità tra due embeddings consecutivi è al di sotto di 0.78 (cambio di argomento) oppure viene raggiunto il limite massimo del numero di frasi per chunk, vengono prese tutte le frasi accumulate fino a quel momento e unite insieme in un'unica variabile, e viene quindi aperto

un nuovo chunk di cui la nuova frase ne sarà la prima a farne parte. Infine, quando il ciclo finisce, l'ultimo chunk viene chiuso e aggiunto alla lista.

```
# CREAZIONE EMBEDDINGS PER DATABASE
def create_db_from_youtube_video_url(video_url):
    video_id = get_video_id_from_url(video_url)
    if not video_id:
        raise ValueError("URL YouTube non valido.")

    # Recuperiamo il titolo del video
    title = get_video_title(video_url)

    # Controllo Cache applicativa
    if video_id in video_database:
        print(f"\n[DATABASE] Il video '{title}' è già presente in memoria.")
        app.config["current_video_id"] = video_id
        return video_id, title

    transcript = get_video_transcript(video_url)
    chunks = create_chunks(transcript)
    embeddings = embedding_model.encode(chunks, convert_to_numpy=True)
    dimension = embeddings.shape[1]

    index = faiss.IndexFlatL2(dimension)
    index.add(embeddings.astype(np.float32))

    # Salviamo includendo il titolo della videolezione
    video_database[video_id] = {
        "title": title,
        "transcript": transcript,
        "chunks": chunks,
        "faiss_index": index
    }
    app.config["current_video_id"] = video_id
    print(f"[DATABASE] Archiviato con successo: '{title}'")
    return video_id, title
```

Passiamo adesso alla creazione del database vettoriale con FAISS: questa è la funzione che viene chiamata non appena l'utente preme il pulsante di invio del link di YouTube dall'interfaccia html. Questa funzione chiama prima le funzioni viste in precedenza di recupero dell'id del video, del titolo, e della trascrizione del video comprendente l'algoritmo di chunking semantico dinamico. Se il video

è già stato elaborato in precedenza il sistema non deve proseguire con la pipeline di calcoli già effettuata ma deve solamente aggiornare l'id del video attivo attuale. Se siamo nel caso in cui il video viene elaborato per la prima volta, la lista dei chunks viene passata al modello multilingual-e5-base che genera un vettore numerico per ogni chunk. Nella variabile "dimension" ci salviamo la lunghezza di un vettore (che per questo modello è fissata a 768 dimensioni numeriche). A questo punto si inizializza con FAISS un indice di tipo distanza euclidea (FlatL2) e i vettori di embeddings appena creati vengono caricati nell'indice FAISS, forzando un casting a float32 poiché FAISS lavora con numeri di questo tipo. Infine i chunks vengono salvati nella RAM all'interno di Flask, in modo da accedervi istantaneamente quando l'utente porrà domande all'LLM. Salviamo inoltre in un dizionario i dati relativi al video (titolo, trascrizione e chunks). Questo permetterà al chatbot di recuperare in maniera rapida le informazioni nel caso l'utente passa da un video all'altro nell'interfaccia.

```
# RECUPERO DEI CHUNK DINAMICO CON FILTRO DI DISTANZA L2
def get_similarity_from_query(query, max_k=RETRIEVAL_MAX_K, distance_threshold=RETRIEVAL_DISTANCE_THRESHOLD, video_id=None):
    # Se il front-end non passa un ID, usare l'ultimo video elaborato
    if video_id is None:
        video_id = app.config.get("current_video_id")

    # Controllo di sicurezza sul dizionario globale che mantiene le informazioni sulle videolezioni
    if not video_id or video_id not in video_database:
        raise ValueError("DB vettoriale non creato o video non trovato. Elabora prima un video.")

    # Estrazione dei dati isolati per la videolezione specifica
    archive_data = video_database[video_id]
    index = archive_data["faiss_index"]
    chunks = archive_data["chunks"]

    query_embedding = embedding_model.encode([query], convert_to_numpy=True)

    distances, indices = index.search(query_embedding.astype(np.float32), max_k)

    retrieved_chunks = []
    print(f"\n--- RICERCA DINAMICA NELL'ARCHIVIO (Video ID: {video_id}) PER: '{query}' ---")

    for i, dist in zip(indices[0], distances[0]):
        if dist <= distance_threshold:
            print(f"Chunk {i} | Distanza: {dist:.4f} -> ACCETTATO")
            retrieved_chunks.append(chunks[i])
        else:
            print(f"Chunk {i} | Distanza: {dist:.4f} -> SCARTATO (Supera {distance_threshold})")

    print(f"Totale chunk dinamici recuperati: {len(retrieved_chunks)} / {max_k}\n")

    return retrieved_chunks
```

Si arriva a questo punto alla fase di retrieval dei chunk pertinenti dal database FAISS, che avviene dopo che l'utente ha posto la query all'LLM, avente l'obiettivo di recuperare solo i chunks della trascrizione che contengono la

risposta. Dopo che l'utente pone la domanda, si va come prima cosa ad identificare il video attivo: dall'id del video la funzione recupera i dati del video (titolo, trascrizione e chunks, fornendo un messaggio di errore se l'utente tenta di fare una domanda al chatbot senza avere inserito un link YouTube. La domanda dell'utente viene passata allo stesso modello di embedding che la trasforma anch'essa in un vettore numerico dello stesso tipo dei chunks. Viene poi avviata la ricerca: FAISS prende il vettore della domanda e ne calcola la distanza euclidea rispetto a tutti i vettori presenti nel database. L'algoritmo esamina per ogni chunk nel db se la sua distanza è inferiore o uguale ad un threshold di 0.415, in tal caso il chunk viene inserito nella lista dei "retrieved_chunks", altrimenti il chunk viene scartato poiché non contiene informazioni rilevanti. Il valore di 0.415 rappresentava un compromesso tra precision e recall, poiché valori minori avrebbero portato ad aumentare troppo la precision, scartando potenzialmente chunks utili per la risposta, mentre valori più alti avrebbero fatto aumentare troppo la recall, recuperando chunks irrilevanti per la risposta. Al termine, la funzione restituisce la lista dei chunk pertinenti per la risposta da dare all'utente.

2.4 Metriche di valutazione

Una volta che l'LLM ha fornito la risposta alla query dell'utente, c'è bisogno di valutare come è stata la sua performance, e in particolare quanto ha rispettato la faithfulness, cioè se l'LLM ha risposto basandosi solamente sulle informazioni del video oppure ha utilizzato le sue conoscenze pregresse, e la answer relevance, cioè se il modello ha risposto alla domanda in modo corretto e non è andato fuori tema. A questo proposito uno dei framework attuali più utilizzati allo scopo è RAGAS (retrieval-augmented generation assessment). Questa tecnica [3] applica il modello LLM-as-a-Judge, dove un LLM viene utilizzato da "giudice", correggendo nel nostro caso le risposte dei nostri LLM locali di LM Studio, e assegnando infine un punteggio compreso tra 0 e 1 per faithfulness e answer relevance. In particolare RAGAS prende la risposta e la scompone in singole affermazioni atomiche, le passa una alla volta al modello chiedendogli se è supportata dal testo del video (risposta si/no), e infine lo score finale equivale al calcolo affermazioni verificate su affermazioni totali.

```

JUDGE_BASE_URL = "http://localhost:1234/v1"
JUDGE_MODEL_NAME = "meta-llama-3.1-8b-instruct"

# COMPONENTI DI GIUDIZIO RAGAS
ragas_llm = LangchainLLMWrapper(ChatOpenAI(
    base_url=JUDGE_BASE_URL,
    api_key="lm-studio",
    model=JUDGE_MODEL_NAME,
    timeout=240,
    max_retries=2
))

ragas_embeddings = LangchainEmbeddingsWrapper(
    HuggingFaceEmbeddings(model_name="intfloat/multilingual-e5-base")
)

# Iniettiamo i nostri modelli locali dentro le metriche RAGAS
faithfulness.llm = ragas_llm
answer_relevancy.llm = ragas_llm
answer_relevancy.embeddings = ragas_embeddings

```

Prima di utilizzare le metriche bisogna configurare correttamente RAGAS: come modello giudice si è optato per meta-llama-3.1-8b-instruct, poichè un buon compromesso tra qualità nel ragionamento e performance del pc, che darà dunque la valutazione alle singole affermazioni generando un punteggio finale. Dopodichè, visto che RAGAS è adatto per funzionare con LangChain e noi invece nel codice utilizziamo il client OpenAI, per facilitare il tutto utilizziamo dei wrapper per convertire questi strumenti nel formato che RAGAS si aspetta di ricevere. Configuriamo poi un timeout di 240 secondi per evitare che, nel caso l'elaborazione spenda troppo tempo, l'LLM locale vada in crash. Configuriamo poi lo stesso modello di embedding (multilingual-e5-base) utilizzato in precedenza che servirà a RAGAS per calcolare le distanze semantiche quando elabora la metrica di answer relevancy, e infine iniettiamo i nostri modelli locali dentro le metriche RAGAS, poichè altrimenti la libreria proverebbe a chiamare i modelli di OpenAI che non stiamo utilizzando.

```

def run_ragas_evaluation(question, answer, contexts):
    data = {
        "question": [question],
        "answer": [answer],
        "contexts": [contexts]
    }
    dataset = Dataset.from_dict(data)

    try:
        print("\n[RAGAS] Calcolo delle metriche di valutazione in corso...")

        configurazione_run = RunConfig(timeout=300)

        result = evaluate(
            dataset=dataset,
            metrics=[faithfulness, answer_relevancy],
            run_config=configurazione_run # <--- INIETTA QUI LA CONFIGURAZIONE
        )

        faith_score = np.mean(result['faithfulness'])
        rel_score = np.mean(result['answer_relevancy'])

        print("REPORT DI VALUTAZIONE RAGAS (Local Judge)")
        print(f"FAITHFULNESS: {faith_score:.4f}")
        print(f"ANSWER RELEVANCY: {rel_score:.4f}")
        return result
    except Exception as e:
        import traceback
        print(f"Impossibile completare la valutazione RAGAS: {e}")
        traceback.print_exc()
        return None

```

Passiamo dunque all'atto di valutazione vero e proprio: questa è la funzione che prende in input una certa interazione con il chatbot e calcola i punteggi numerici di faithfulness e answer relevance. Per la chat socratica non sono stati effettuati i test poiché non gli LLM non sono riusciti ad implementarla, dunque i test sono svolti sulla modalità di chat standard. Poiché RAGAS richiede una struttura dati basata sulla libreria "datasets", creiamo un oggetto inserendogli la domanda dell'utente, la risposta dell'LLM e i chunk estratti dal db FAISS usati come base per la risposta. Configuriamo un timeout di RAGAS di 300 secondi, poiché visto che RAGAS richiede che l'LLM giudice analizzi i testi e esegua diversi passaggi, questo in locale può richiedere tempo e dunque un timeout serve proprio ad

evitare che RAGAS interrompa il processo prima che il giudice abbia restituito i valori finali delle metriche. A questo punto con la funzione `evaluate()`, la principale del framework RAGAS, si prende in input l'oggetto dataset costruito in precedenza e le due metriche da valutare (*faithfulness* e *answer relevancy*) e avvia la valutazione. Poiché RAGAS calcola un punteggio per ogni singola affermazione atomica all'interno della risposta, viene calcolata la media di queste, corrispondente al punteggio finale della metrica in questione.

Abbiamo calcolato i valori di queste metriche relativamente agli stessi video utilizzati in precedenza per il fine-tuning della *similarity threshold*: in particolare per ogni video e per ognuno dei tre LLM utilizzati come chatbot abbiamo posto 3 domande per ognuna delle seguenti 3 categorie:

- domande pertinenti (*domande di cui la risposta è presente nel testo della trascrizione*);
- domande dello stesso contesto ma non pertinenti (*domande che trattano dello stesso argomento del video, ma di cui nello stesso non è contenuta la risposta*);
- domande non pertinenti (*domande relative a contesti differenti dal video*);

In particolare, per la prima categoria, dove **la risposta è contenuta nella trascrizione**, sono state poste le seguenti domande:

TELEGIORNALE
Quanti uomini sono scomparsi a Pesaro?
Da chi è nata l'idea di fare un documentario sulla battaglia del Metauro?
Qual è l'idea di infrastrutture prevista da RFI?

DOCUMENTARIO
In che periodo Adolf Hitler fu nominato cancelliere?
Come era composta la <i>Vermacht</i> nel 1939?
Chi erano i pirati dell'Edel?

LEZIONE UNIVERSITÀ
Cos'è un insieme limitato superiormente?
In quanti modi si possono ordinare n oggetti?
Se moltiplichiamo due numeri razionali cosa otteniamo?

SPETTACOLO
Quali sono i piatti tipici dell'autogrill?
Cosa ha fatto il principe azzurro quando è arrivato a cavallo?
A quale pittore hanno rubato la tela?

HIGHLIGHTS PARTITA DI CALCIO
Come è finita la conclusione di Raspadori?
Quanti gol ha fatto Lukaku nel campionato?
Dopo quanto tempo il Napoli è tornato a vincere il campionato?

Per la categoria in cui **il contesto è lo stesso ma la risposta non è contenuta nel video**, le domande scelte sono le seguenti:

TELEGIORNALE
Esiste un termine minimo entro il quale denunciare nel caso una persona scompaia?
Quanti incidenti sul lavoro capitano ogni giorno in Campania?
Quali sono le normative sulla mobilità sostenibile?

DOCUMENTARIO
Come si chiamava l'esercito della Germania Ovest?
Quale è stato il ruolo della Cina nella Seconda guerra mondiale?
In che modo il bolscevismo ha influenzato gli anni della guerra fredda?

LEZIONE UNIVERSITÀ
Chi è stato il primo popolo ad utilizzare i numeri razionali?
Chi ha dimostrato l'irrazionalità di radice di due?
Quale è la dimostrazione del teorema di Fermat?

SPETTACOLO
Cosa ha fatto cappuccetto rosso?
Dove nasce la figura del principe azzurro?

Chi sono i personaggi principali del teatro napoletano?

HIGHLIGHTS PARTITA DI CALCIO
In che squadra giocava Lukaku prima di arrivare a Napoli?
Quanto è finita Inter-Napoli dell'anno 2024/2025?
Quanti sono i calciatori che scendono in campo in una partita?

Infine, le domande **fuori dal contesto** sono sempre le stesse per ognuno dei video:

DOMANDE NON PERTINENTI
Quale è la capitale della Francia?
Quanti geni ha il DNA di un essere umano?
Quale è la stella più vicina alla Terra dopo il Sole?

Abbiamo dunque calcolato le metriche per tali domande e per ogni LLM utilizzato (gemma-2-9b-it, qwen2.5-1.5b-instruct e meta-llama-3.1-8b-instruct) e utilizzando come modello giudice meta-llama-3.1-8b-instruct, con i risultati riportati nelle tabelle sottostanti.

Di seguito le tabelle dei risultati ottenuti utilizzando Qwen:

TELEGIORNALE

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	quanti uomini sono scomparsi a Pesaro?	Non lo so.	0,0000	0,0000
2	DOMANDE PERTINENTI	da chi è nata l'idea di fare un documentario sulla battaglia del Metauro?	Non lo so.	0,0000	0,7578
3	DOMANDE PERTINENTI	Quale è l'idea di infrastrutture prevista da RFI?	Non lo so.	N/A	0,0000
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	Esiste un termine minimo entro il quale denunciare nel caso una persona scompaia?	Non lo so.	0,0000	0,0000
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quanti incidenti sul lavoro capitano ogni giorno in Campania?	Non lo so.	N/A	0,0000
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quali sono le normative sulla mobilità sostenibile?	Non lo so.	N/A	0,0000
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	0,0000	0,0000
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?	Non lo so.	0,0000	0,0000
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?	Non lo so.	0,0000	0,0000
MEDIA TOTALE				0,0000	0,0842

DOCUMENTARIO

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	In che periodo Adolf Hitler fu nominato cancelliere?	Nel gennaio del 1933.	N/A	0,7481
2	DOMANDE PERTINENTI	Come era composta la Wehrmacht nel 1939?	Non lo so.	0,0000	0,0000
3	DOMANDE PERTINENTI	Chi erano i pirati dell'Edel?	Non lo so.	1,0000	0,0000
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	Come si chiamava l'esercito della Germania Ovest?	Non lo so.	0,0000	0,0000
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quale è stato il ruolo della Cina nella seconda guerra mondiale?	Non lo so.	0,0000	0,7530
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	In che modo il bolscevismo ha influenzato gli anni della guerra fredda?	Non lo so.	0,0000	0,0000
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	0,0000	0,0000
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?	Non lo so.	0,0000	0,0000
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?	Non lo so.	0,5000	0,0000
MEDIA TOTALE				0,1875	0,1668

LEZIONE UNIVERSITARIA

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	Cos'è un insieme limitato superiormente?	Non lo so.	0,0000	0,0000
2	DOMANDE PERTINENTI	In quanti modi si possono ordinare n oggetti?	Non lo so.	0,0000	0,0000
3	DOMANDE PERTINENTI	Se moltiplichiamo due numeri razionali cosa otteniamo?	Non lo so.	0,0000	0,0000
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	Chi è stato il primo popolo ad utilizzare i numeri razionali?	Non lo so.	0,0000	0,0000
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Chi ha dimostrato l'irrazionalità di radice di due?	Non lo so.	0,0000	0,0000
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quale è la dimostrazione del teorema di Fermat?	Non lo so.	0,0000	0,7759
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	0,0000	0,7681
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?	Non lo so.	0,5000	0,0000
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?	Non lo so.	N/A	0,0000
MEDIA TOTALE				0,0625	0,1715

SPETTACOLO

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	quali sono i piatti tipici dell'autogrill?	Non lo so.	0,0000	0,0000
2	DOMANDE PERTINENTI	Cosa ha fatto il principe azzurro quando è arrivato a cavallo?	Non lo so.	0,0000	0,0000
3	DOMANDE PERTINENTI	A quale pittore hanno rubato la tela?	Non lo so.	0,0000	0,7493
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	Cosa ha fatto cappuccetto rosso?	Non lo so.	0,0000	0,7700
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Dove nasce la figura del principe azzurro?	Non lo so.	0,5000	0,0000
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	Chi sono i personaggi principali del teatro napoletano?	Non lo so.	0,0000	0,0000
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	0,0000	0,0000
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?	Non lo so.	0,0000	0,0000
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?	Non lo so.	0,0000	0,0000
MEDIA TOTALE				0,0556	0,1688

HIGHLIGHTS PARTITA

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	Come è finita la conclusione di Raspadori?	Non lo so.	1,0000	0,0000
2	DOMANDE PERTINENTI	Quanti gol ha fatto Lukaku nel campionato?	14	1,0000	0,7249
3	DOMANDE PERTINENTI	Dopo quanto tempo il Napoli ha tornato a vincere il campionato?	Non lo so.	0,5000	0,0000
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	In che squadra giocava Lukaku prima di arrivare a Napoli?	Non lo so.	N/A	0,7562
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quanto è finita Inter-Napoli dell'anno 2024/2025?	Non lo so.	N/A	0,0000
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quanti sono i calciatori che scendono in campo in una partita?	Non lo so.	0,5000	0,7735
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	0,0000	0,0000
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?	Non lo so.	0,0000	0,0000
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?	Non lo so.	0,5000	0,0000
MEDIA TOTALE				0,5000	0,2505

Di seguito i risultati delle metriche utilizzando come chatbot Llama:

TELEGIORNALE

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	quanti uomini sono scomparsi a Pesaro?	Un uomo.	1,0000	0,7244
2	DOMANDE PERTINENTI	da chi è nata l'idea di fare un documentario sulla battaglia del Metauro?	L'idea di fare un documentario sulla battaglia del Metauro è nata dalle ricerche del professor Oscarmeì, un caro amico e archeologo dell'Università di Urbino.	0,3333	0,9846
3	DOMANDE PERTINENTI	Quale è l'idea di infrastrutture prevista da RFI?	La visione di RFI per il territorio è quella di realizzare delle infrastrutture che siano inserite nel contesto rispettoso del contesto socioeconomico, del contesto	0,6667	0,8810
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	Esiste un termine minimo entro il quale denunciare nel caso una persona scompaia?	Non lo so.	0,0000	0,7927
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quanti incidenti sul lavoro capitano ogni giorno in Campania?	Non lo so.	N/A	0,7609
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quali sono le normative sulla mobilità sostenibile?	Non lo so.	0,0000	0,0000
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	0,5000	0,0000
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?	Non lo so.	0,0000	0,7893
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?	Non lo so.	0,0000	0,0000
MEDIA TOTALE				0,3125	0,5481

DOCUMENTARIO

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	In che periodo Adolf Hitler fu nominato cancelliere?	Il gennaio del 1933.	N/A	0,7879
2	DOMANDE PERTINENTI	Come era composta la Wehrmacht nel 1939?	La Wehrmacht era composta da circa 1 milione e mezzo di uomini, quasi 2500 carri armati e più di 2000 aerei pronti a partecipare all'offensiva. Inoltre, la Wehrmacht era composta da diversi gruppi giovanili che rifiutavano la disciplina di partito e l'indottrinamento obbligatorio. Non erano organizzati in una struttura	1,0000	0,7906
3	DOMANDE PERTINENTI	Chi erano i pirati dell'Edel?	I cosiddetti "pirati dell'Edel" erano gruppi giovanili che rifiutavano la disciplina di partito e l'indottrinamento obbligatorio. Non erano organizzati in una struttura	N/A	0,9556
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	Come si chiamava l'esercito della Germania Ovest?	Non lo so.	0,0000	0,0000
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quale è stato il ruolo della Cina nella seconda guerra mondiale?	Non lo so.	N/A	0,0000
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	In che modo il bolscevismo ha influenzato gli anni della guerra fredda?	Non lo so.	0,5000	0,0000
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	0,0000	0,7681
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?	Non lo so.	0,0000	0,0000
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?	Non lo so.	N/A	0,0000
MEDIA TOTALE				0,3000	0,3669

LEZIONE UNIVERSITARIA

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	Cos'è un insieme limitato superiormente?	Non lo so.	0,0000	0,0000
2	DOMANDE PERTINENTI	In quanti modi si possono ordinare n oggetti?	Non lo so.	N/A	0,0000
3	DOMANDE PERTINENTI	Se moltiplichiamo due numeri razionali cosa otteniamo?	Non lo so.	0,0000	0,0000
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	Chi è stato il primo popolo ad utilizzare i numeri razionali?	Non lo so.	N/A	0,0000
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Chi ha dimostrato l'irrazionalità di radice di due?	Non lo so.	0,0000	0,0000
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quale è la dimostrazione del teorema di fermat?	Non lo so.	0,0000	0,0000
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	0,0000	0,0000
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?	Non lo so.	0,0000	0,0000
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?	Non lo so.	0,0000	0,0000
MEDIA TOTALE				0,0000	0,0000

SPETTACOLO

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	quali sono i piatti tipici dell'autogrill?	Non lo so.	N/A	0,0000
2	DOMANDE PERTINENTI	Cosa ha fatto il principe azzurro quando è arrivato a cavallo?	Non lo so.	0,0000	0,0000
3	DOMANDE PERTINENTI	A quale pittore hanno rubato la tela?	Non lo so.	0,0000	0,0000
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	Cosa ha fatto cappuccetto rosso?	Non lo so.	0,0000	0,0000
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Dove nasce la figura del principe azzurro?	Non lo so.	0,0000	0,0000
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	Chi sono i personaggi principali del teatro napoletano?	Non lo so.	0,0000	0,0000
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	0,0000	0,0000
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?	Non lo so.	0,0000	0,0000
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?	Non lo so.	N/A	0,0000
MEDIA TOTALE				0,0000	0,0000

HIGHLIGHTS PARTITA

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	Come è finita la conclusione di Raspadori?	La conclusione di Raspadori è finita larga.	0,3333	0,9699
2	DOMANDE PERTINENTI	Quanti gol ha fatto Lukaku nel campionato?	Lukaku ha fatto 14 gol nel campionato.	N/A	1,0000
3	DOMANDE PERTINENTI	Dopo quanto tempo il Napoli ha tornato a vincere il campionato?	Il Napoli è tornato a vincere il campionato 2 anni dopo Spalletti con Antonio Conte.	0,6667	0,8812
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	In che squadra giocava Lukaku prima di arrivare a Napoli?	Non lo so.	N/A	0,0000
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quanto è finita Inter-Napoli dell'anno 2024/2025?	Non lo so.	0,0000	0,7650
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quanti sono i calciatori che scendono in campo in una partita?	Non lo so.	0,0000	0,0000
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	1,0000	0,7681
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?	Non lo so.	N/A	0,0000
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?	Non lo so.	0,5000	0,7745
MEDIA TOTALE				0,4167	0,5732

Di seguito i risultati delle metriche ottenute con LLM Gemma:

TELEGIORNALE

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	quanti uomini sono scomparsi a Pesaro?	L'idea di fare un documentario sulla battaglia del Metauro è nata dal professor Oscarmeì.	0,5000	0,0000
2	DOMANDE PERTINENTI	da chi è nata l'idea di fare un documentario sulla battaglia del Metauro?		0,0000	0,9136
3	DOMANDE PERTINENTI	Quale è l'idea di infrastrutture prevista da RFI?	La visione di RFI e quella di realizzare delle infrastrutture inserite nel loro contesto, rispettose del contesto socioeconomico, del contesto territoriale e	0,5000	0,0000
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	Esiste un termine minimo entro il quale denunciare nel caso una persona scompaia?		0,0000	0,0000
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quanti incidenti sul lavoro capitano ogni giorno in Campania?	Non lo so.	0,0000	0,0000
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quali sono le normative sulla mobilità sostenibile?		0,3333	0,0000
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	0,0000	0,0000
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?		0,5000	0,0000
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?	Non lo so.	0,5000	0,0000
MEDIA TOTALE				0,2593	0,1015

DOCUMENTARIO

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	In che periodo Adolf Hitler fu nominato cancelliere?	Gennaio del 1933.	0,5000	0,9842
2	DOMANDE PERTINENTI	Come era composta la Wehrmacht nel 1939?	La Wehrmacht nel 1939 era composta da diversi milioni di uomini, dotata di divisioni moderne, una grande aviazione e una marina in espansione. I pirati dell'Edel erano gruppi giovanili che imitavano la disciplina di partito e l'indottrinamento obbligatorio.	N/A	0,7844
3	DOMANDE PERTINENTI	Chi erano i pirati dell'Edel?		1,0000	1,0000
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	Come si chiamava l'esercito della Germania Ovest?	Wehrmacht	N/A	0,0000
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quale è stato il ruolo della Cina nella seconda guerra mondiale?		0,0000	0,0000
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	In che modo il bolscevismo ha influenzato gli anni della guerra fredda?		N/A	0,0000
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	0,0000	0,0000
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?	Non lo so.	0,0000	0,0000
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?	Non lo so.	0,0000	0,0000
MEDIA TOTALE				0,2500	0,3076

LEZIONE UNIVERSITARIA

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	Cos'è un insieme limitato superiormente?	Non lo so.	0,0000	0,0000
2	DOMANDE PERTINENTI	In quanti modi si possono ordinare n oggetti?	Non lo so.	0,0000	0,0000
3	DOMANDE PERTINENTI	Se moltiplichiamo due numeri razionali cosa otteniamo?		0,0000	0,0000
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	Chi è stato il primo popolo ad utilizzare i numeri razionali?	Non lo so.	0,5000	0,0000
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Chi ha dimostrato l'irrazionalità di radice di due?	Non lo so.	0,5000	0,0000
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quale è la dimostrazione del teorema di fermat?		0,0000	0,0000
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	0,0000	0,0000
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?		0,0000	0,0000
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?		0,5000	0,0000
MEDIA TOTALE				0,1667	0,0000

SPETTACOLO

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	quali sono i piatti tipici dell'autogrill?	Non lo so.	0,0000	0,0000
2	DOMANDE PERTINENTI	Cosa ha fatto il principe azzurro quando è arrivato a cavallo?	Non lo so.	0,5000	0,0000
3	DOMANDE PERTINENTI	A quale pittore hanno rubato la tela?	Non lo so.	0,0000	0,0000
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	Cosa ha fatto cappuccetto rosso?	Non lo so.	0,0000	0,0000
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Dove nasce la figura del principe azzurro?	Non lo so.	0,0000	0,0000
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	Chi sono i personaggi principali del teatro napoletano?	Non lo so.	0,0000	0,0000
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	0,0000	0,0000
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?		0,0000	0,0000
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?		0,5000	0,0000
MEDIA TOTALE				0,1111	0,0000

HIGHLIGHTS PARTITA

ID	CATEGORIA DOMANDA	DOMANDA	RISPOSTA CHATBOT (QWEN)	FAITHFULNESS	ANSWER RELEVANCY
1	DOMANDE PERTINENTI	Come è finita la conclusione di Raspadori?		1,0000	0,0000
2	DOMANDE PERTINENTI	Quanti gol ha fatto Lukaku nel campionato?	14	1,0000	0,0000
3	DOMANDE PERTINENTI	Dopo quanto tempo il Napoli ha tornato a vincere il campionato?	Due anni.	0,5000	0,0000
4	DOMANDE DEL CONTESTO MA NON PERTINENTI	In che squadra giocava Lukaku prima di arrivare a Napoli?		0,0000	0,0000
5	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quanto è finita Inter-Napoli dell'anno 2024/2025?	Non lo so.	1,0000	0,0000
6	DOMANDE DEL CONTESTO MA NON PERTINENTI	Quanti sono i calciatori che scendono in campo in una partita?	Non lo so.	0,0000	0,0000
7	DOMANDE NON PERTINENTI	Quale è la capitale della Francia?	Non lo so.	0,0000	0,0000
8	DOMANDE NON PERTINENTI	Quanti geni ha il DNA di un essere umano?	Non lo so.	0,0000	0,0000
9	DOMANDE NON PERTINENTI	Quale è la stella più vicina alla Terra dopo il Sole?	Non lo so.	0,5000	0,0000
MEDIA TOTALE				0,4444	0,0000

2.5 Gestione delle routes e interfaccia html

In questo paragrafo vediamo come è fatta l'interfaccia html e le sue "routes", cioè come le funzionalità dell'interfaccia sono collegate alle funzioni in python viste in precedenza.

```
# INTERFACCIA HTML
@app.route('/')
def index():
    return render_template('index.html')
```

Questa è la rotta principale dell'applicazione web Flask. In particolare dice cosa succede quando l'utente digita l'indirizzo del sito nel browser (<http://localhost:1234>). In questo caso Flask va a cercare il file index.html e lo carica nel browser dell'utente.

```
<!-- MENU DI SELEZIONE VIDEOLEZIONE -->
<div class="menu-card">
  <h2>📺 Videolezione Attiva</h2>
  <select id="videoSelect" onchange="switchVideo()">
    <option value="">-- Nessuna videolezione caricata. Elabora un link sotto per iniziare --</option>
  </select>
</div>
```

Qui invece abbiamo l'interfaccia utente del selettore del video da un menu: quando un utente sceglie un video tra quelli che ha caricato viene attivata la funzione switchVideo() descritta sotto che comunicherà a Flask di aggiornare il contesto con il nuovo video selezionato. Di seguito il codice:

```
// --- CAMBIO VIDEO DAL MENU ---
function switchVideo() {
    const select = document.getElementById('videoSelect');
    if (!select) {
        console.warn('[RAG Warning] Impossibile cambiare video: "videoSelect" non trovato.');
```

return;

```
    }
    const selectedId = select.value;
    const chatContainer = document.getElementById('chatResultContainer');
    // Salviamo lo stato della chat precedente
    if (currentVideoId && loadedVideos[currentVideoId] && chatContainer) {
        loadedVideos[currentVideoId].chatHTML = chatContainer.innerHTML;
    }
    currentVideoId = selectedId;
    if (!currentVideoId) {
        toggleUI(false);
        if (chatContainer) chatContainer.innerHTML = '';
        const searchRes = document.getElementById('searchResultsContainer');
        if (searchRes) searchRes.innerHTML = '';
        return;
    }
    toggleUI(true);
    // Ripristina la chat del nuovo video selezionato
    if (chatContainer) {
        chatContainer.innerHTML = loadedVideos[currentVideoId].chatHTML || '';
    }
    // Pulisce i vecchi risultati di ricerca chunk
    const searchRes = document.getElementById('searchResultsContainer');
    if (searchRes) searchRes.innerHTML = '';
    const searchStatus = document.getElementById('searchStatus');
    if (searchStatus) searchStatus.innerHTML = '';
    console.log(`Contesto aggiornato sulla lezione: ${currentVideoId}`); }
```

Prima di cambiare il video, la funzione recupera il video corrente attivo e salva i messaggi scambiati con l’LLM nella memoria RAM in modo da non perderli per un cambio di video successivo. Se l’utente seleziona il placeholder iniziale la pagina viene resettata e ritorna come in partenza, mentre se l’utente ha selezionato un video già attivo, la funzione recupera l’HTML della chat precedente e la ricarica nell’interfaccia.

```
<!-- INTERFACCIA INSERIMENTO LINK YOUTUBE -->
<div class="card">
  <h2>1. Inizializza il Sistema (Inserisci Video)</h2>
  <p>Incolla l'URL di un video YouTube per scaricare la trascrizione e creare il Database Vettoriale locale (FAISS).</p>
  <div class="flex-row">
    <input type="text" id="videoUrl" placeholder="https://www.youtube.com/watch?v=...">
    <button id="btnProcess" onclick="processVideo()">Elabora</button>
  </div>
  <span id="processSpinner" class="spinner"> 🔄 Vettorizzazione in corso </span>
  <div id="processStatus" class="status"></div>
</div>
```

Dopo aver creato gli elementi CSS necessari all'inizio del codice, vediamo i frammenti di codice corrispondenti agli elementi principali dell'interfaccia. Il codice in figura rappresenta l'interfaccia di inserimento del link youtube che appare all'utente all'inizio della pagina html. Consiste nel campo di testo in cui l'utente incolla il link del video YouTube e il pulsante di invio al cui clic verrà eseguita la seguente funzione JavaScript processVideo():

```
// --- ELABORA ED ARCHIVIA VIDEO ---
async function processVideo() {
    const urlInput = document.getElementById('videoUrl');
    const url = urlInput ? urlInput.value.trim() : '';
    const statusDiv = document.getElementById('processStatus');
    const spinner = document.getElementById('processSpinner');
    const btnProcess = document.getElementById('btnProcess');
    if (!url) {
        if (statusDiv) statusDiv.innerHTML = '<span class="error">Inserisci un URL valido.</span>';
        return;
    }
    if (statusDiv) statusDiv.innerHTML = '';
    if (spinner) spinner.style.display = 'inline';
    if (btnProcess) btnProcess.disabled = true;
    try {
        const response = await fetch('/process_video', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ url: url })
        });
        const data = await response.json();
        if (response.ok) {
            if (statusDiv) statusDiv.innerHTML = `<span class="success">${data.message}</span>`;
            const videoId = data.video_id;
            const videoTitle = data.title;

            if (!loadedVideos[videoId]) {
                loadedVideos[videoId] = {
                    url: url,
                    title: videoTitle,
                    chatHTML: ""
                };
            }
        }
    }
};
```

```

const select = document.getElementById('videoSelect');
if (select) {
    // Svuota il messaggio placeholder di default al caricamento del primo video
    if (Object.keys(loadedVideos).length === 1) {
        select.innerHTML = '';
    }
    // Creiamo il nuovo elemento del menu usando il titolo del video
    const option = document.createElement('option');
    option.value = videoId;
    option.textContent = `${videoTitle}`;
    select.appendChild(option);
}
}
const select = document.getElementById('videoSelect');
if (select) select.value = videoId;
switchVideo();
if (urlInput) urlInput.value = '';
} else {
    // Se il backend risponde con un errore (es. sottotitoli mancanti), lo stampa chiaramente a schermo
    if (statusDiv) statusDiv.innerHTML = `<span class="error">Errore: ${data.error}</span>`;
}
} catch (error) {
    if (statusDiv) statusDiv.innerHTML = `<span class="error">Errore di connessione: ${error}</span>`;
    console.error(error);
} finally {
    if (spinner) spinner.style.display = 'none';
    if (btnProcess) btnProcess.disabled = false;
}
}

```

Il codice viene dunque eseguito quando l'utente preme sul pulsante di invio dopo aver incollato il link YouTube. Come prima cosa la funzione va a recuperare gli elementi HTML (il campo dell'URL, il div di stato, lo spinner e il pulsante), e con la funzione trim() estrae l'URL digitato dall'utente rimuovendo eventuali caratteri vuoti all'inizio e alla fine. Nel caso non vede un URL si stampa un messaggio di errore. Subito dopo viene modificata l'interfaccia grafica: l'icona di caricamento si rende visibile, il pulsante appena cliccato viene disabilitato per impedire che l'utente ci clicchi sopra più volte. Si invia a questo punto una richiesta HTTP di tipo POST al server Flask e si attende che questo finisca di scaricare e vettorizzare il video. La risposta finale verrà salvata in un oggetto JavaScript. Se il tutto va a buon fine lo stato iniziale dell'interfaccia viene ripristinato. Se il video è nuovo, i suoi dati vengono salvati in una variabile globale utilizzando come chiave il suo id e viene inizializzata la sua chat, inizialmente vuota. Se è il primo video caricato, viene rimossa dall'interfaccia la scritta "Nessuna videolezione inserita". Viene poi generato un elemento HTML in cui viene reso visibile il titolo del video corrente. Infine il valore del menu viene posto all'id del video appena caricato.

```

# ROUTE QUANDO L'UTENTE PREME INVIA DOPO AVER INSERITO IL LINK DEL VIDEO YOUTUBE
@app.route(
    '/process_video',
    methods=['POST']
)
def process_video():
    data = request.json
    url = data.get('url')
    if not url:
        return jsonify(
            {
                'error': 'URL mancante'
            }
        ), 400
    try:
        create_db_from_youtube_video_url(url)
        return jsonify(
            {
                'message': 'Video elaborato. RAG pronto.'
            }
        )
    except Exception as e:
        return jsonify(
            {
                'error': str(e)
            }
        ), 500

```

La richiesta a Flask dopo che l'utente preme sul pulsante di elaborazione del link YouTube porta al codice di questa route, che ha il compito di elaborare le informazioni e restituire una risposta in formato JSON. In dettaglio, come primo passo la route viene definita con il metodo POST, indicando che se qualcuno provasse a digitare l'indirizzo nella barra del browser non verrebbe eseguito (poiché il browser esegue richieste GET). Con `request.json` i dati inviati da JavaScript vengono ottenuti da Flask e viene estratto l'URL. Si chiama infine la funzione di creazione del db vettoriale già vista e restituisce una risposta JSON, gestendo sempre in maniera tempestiva il caso in cui accadano errori.

Passiamo adesso alla descrizione dello stesso asse HTML-JavaScript-Flask nel caso della chat con l'LLM:

```

<!-- INTERFACCIA CHATBOT -->
<div class="card">
  <h2>3. Interroga l'LLM (Assistente)</h2>
  <p>Scegli lo stile di conversazione e fai una domanda sul video.</p>

  <!-- RADIO BUTTON PER SCELTA MODALITA' CHAT -->
  <div class="radio-group">
    <label>
      <input type="radio" name="chatMode" value="standard" checked> 🤖 Assistente Standard
    </label>
    <label>
      <input type="radio" name="chatMode" value="socratic"> 🏛️ Dialogo Socratico
    </label>
  </div>

  <div class="flex-row">
    <input type="text" id="chatQuery" placeholder="Fai una domanda sul contenuto del video..." disabled>
    <button id="btnChat" onclick="askChatbot()" disabled>Chiedi all'AI</button>
  </div>
  <div id="chatResultContainer"></div>
</div>

```

Il tutto, racchiuso in una sezione div, inizia con la scelta attraverso un radio button della modalità di interazione con il chatbot, standard o socratica, di default selezionata a standard, a seconda della quale Flask poi utilizzerà il corrispondente prompt, SYSTEM_PROMPT per la chat standard e SOCRATIC_prompt per la seconda modalità. Si crea poi il campo di testo per porre la domanda e il pulsante di cui al click si attiverà la funzione JavaScript askChatbot(). Queste due componenti sono sbloccati una volta che l'utente inserisce il link del video, cioè una volta che la route precedente process_video verrà eseguita con successo. Si prepara infine un ultimo contenitore div che conterrà di volta in volta le domande poste e le risposte ricevute.

```

async function askChatbot() {
    const input = document.getElementById('chatQuery');
    const query = input.value.trim();
    if (!query) return;
    const container = document.getElementById('chatResultContainer');
    const mode = document.querySelector('input[name="chatMode"]:checked').value;
    const modeBadge = mode === 'socratic' ? '🔗 [Soc]' : '📄 [Std]';
    container.innerHTML += `<div class="user-msg"><b>Tu ${modeBadge}</b> ${query}</div>`;
    input.value = '';
    // CREAZIONE SKELETON PER ATTESA DELLE RISPOSTE
    const skeletonId = 'skeleton-' + Date.now();
    container.innerHTML += `
        <div id="${skeletonId}" class="bot-msg">
            <b>Assistente:</b>
            <div class="skeleton" style="width: 75%;"></div>
            <div class="skeleton" style="width: 50%;"></div>
        </div>`;
    container.scrollTop = container.scrollHeight;
    const formData = new FormData();
    formData.append('query', query);
    formData.append('session_id', 'user_123');
    formData.append('mode', mode);
    try {
        const response = await fetch('/chatbot', { method: 'POST', body: formData });
        const reply = await response.text();
        // RIMOZIONE DELLO SKELETON E VISIONE DELLA RISPOSTA
        document.getElementById(skeletonId).remove();
        container.innerHTML += `<div class="bot-msg"><b>Assistente:</b> ${reply}</div>`;
    } catch (error) {
        document.getElementById(skeletonId).remove();
        container.innerHTML += `<div class="error">❌ Errore durante la generazione.</div>`;
    } finally {
        container.scrollTop = container.scrollHeight;
    }
}

```

Questa è la funzione che si attiva ogni volta che l'utente pone una domanda all'LLM: come prima cosa la funzione estrae il testo inserito dall'utente rimuovendo eventuali spazi vuoti all'inizio e alla fine con `trim()` e bloccando l'esecuzione nel caso l'utente inserisce un prompt vuoto. Si rileva infine la modalità di chat (standard o socratica). Viene aggiunto un badge visivo per visualizzare il tipo di chat e viene svuotato il campo di testo per essere pronto per una nuova domanda. Per attendere la risposta viene mostrato nell'interfaccia un effetto skeleton, cioè delle barre grigie animate per indicare che il testo di risposta dell'LLM è in fase di caricamento, implementando uno scorrimento verso il basso nel caso non ci troviamo nella prima domanda ma la chat prosegue. Vengono poi impacchettati i dati (la query dell'utente, l'ID della sessione e la modalità di chat) e inviati come richiesta POST al server Flask e si

attende che l'LLM locale generi la risposta. Una volta effettivamente arrivata, viene rimosso lo skeleton di caricamento della risposta e viene inserita al suo posto la risposta generata dal chatbot.

```
chat_history = {}

# ROUTE QUANDO L'UTENTE EFFETTUA UNA DOMANDA AL LLM
@app.route('/chatbot', methods=['POST'])
def chatbot():
    session_id = request.form.get('session_id', 'default_user')
    user_query = request.form['query']
    mode = request.form.get('mode', 'standard')

    # Inizializzazione storico
    if session_id not in chat_history:
        chat_history[session_id] = []

    current_system_prompt = SOCRATIC_PROMPT if mode == 'socratic' else SYSTEM_PROMPT

    # Recupero dinamico dei chunk
    retrieved_chunks = get_similarity_from_query(user_query)

    if not retrieved_chunks:
        bot_reply = "Non lo so."
        chat_history[session_id].append({"role": "user", "content": user_query})
        chat_history[session_id].append({"role": "assistant", "content": bot_reply})
        return bot_reply

    # -----

    retrieved_context = "\n\n".join(retrieved_chunks)
```

```

# COSTRUZIONE DEL MESSAGGIO CORRETTA CON DELIMITATORI
augmented_prompt = (
    f"Usa le informazioni contenute nel seguente contesto per rispondere alla domanda finale.\n\n"
    f"### INIZIO CONTESTO ###\n"
    f"{retrieved_context}\n"
    f"### FINE CONTESTO ###\n\n"
    f"DOMANDA: {user_query}"
)

messages = [{"role": "system", "content": current_system_prompt}]
messages.extend(chat_history[session_id][-4:])
messages.append({"role": "user", "content": augmented_prompt})

# GESTIONE ERRORI SULLA CHIAMATA LLM
try:
    response = client.chat.completions.create(
        model=MODEL_NAME,
        messages=messages,
        temperature=0.4 if mode == 'socratic' else 0.1,
        max_tokens=600
    )
    bot_reply = response.choices[0].message.content

except Exception as e:
    print(f"Errore durante la generazione LLM: {e}")
    return "Si è verificato un errore di connessione con il modello locale. Controlla il terminale."

# Salvataggio nello storico
chat_history[session_id].append({"role": "user", "content": user_query})
chat_history[session_id].append({"role": "assistant", "content": bot_reply})

# VALUTAZIONE RAGAS
run_ragas_evaluation(user_query, bot_reply, retrieved_chunks)
return bot_reply

```

I dati impacchettati (query, sessione, modalità) vengono spediti a questa route dove per prima cosa li va a recuperare. Viene creato un oggetto `chat_history()` per conservare i messaggi della chat corrente. Viene verificata poi la modalità e viene scelto di conseguenza il tipo di prompt già visto in precedenza (standard o socratico). Viene a questo punto invocata la funzione del retrieval dei chunks vista in precedenza per la ricerca dei chunks pertinenti da utilizzare per la risposta. Se nessun chunk viene recuperato l'LLM dovrà rispondere "Non lo so". Se invece un certo numero di chunks vengono recuperati, il testo viene unito in un'unica stringa per strutturare il prompt finale da cui l'LLM deve prendere le informazioni per rispondere. Per evitare di rallentare l'LLM si è previsto di recuperare solo gli ultimi 4 messaggi della chat (le ultime 2 domande e 2 risposte) a cui si appendono la query attuale dell'utente insieme ai chunks recuperati e ai vari prompt di fine-tuning da dare all'LLM. Nel caso della chat

socratica si utilizza una temperatura di 0.4 per consentire di variare la struttura linguistica della risposta, mentre nel caso della chat standard il valore viene abbassato a 0.1 poiché vogliamo che il LLM si basi esclusivamente sul testo dei chunks recuperati, senza variarne il linguaggio. Una volta ottenuta la risposta, questa viene salvata nello storico dei messaggi e vengono avviate le metriche di valutazione RAGAS.

2.6 Interfaccia HTML

Di seguito l'aspetto visivo dell'interfaccia visibile all'utente:

Assistente Universitario RAG

Videolezione Attiva

– Nessuna videolezione caricata. Elabora un link sotto per iniziare –

Inizializza il Sistema (Inserisci url video)

Incolla l'URL di un video YouTube per scaricare la trascrizione e aggiungerlo all'archivio dell'applicazione.

Elabora ed Archivia

Interroga l'LLM (Assistente)

Scegli lo stile di conversazione e fai una domanda. La chat è isolata e legata solo alla lezione attiva.

☒ Assistente Standard ☐ Dialogo Socratico

Chiedi all'AI

Inizialmente l'interfaccia si presenta in questo modo, con nessun video ancora caricato dall'utente.

Videolezione Attiva

 Radiostuni News del 27/06/2026 

Inizializza il Sistema (Inserisci url video)

Incolla l'URL di un video YouTube per scaricare la trascrizione e aggiungerlo all'archivio dell'applicazione.

Elabora ed
Archivia

✓ Video elaborato e indicizzato con successo.

Interroga l'LLM (Assistente)

Scegli lo stile di conversazione e fai una domanda. La chat è isolata e legata solo alla lezione attiva.

☒  Assistente Standard ☐  Dialogo Socratico

Chiedi
all'AI

Tu  [Std]: Quando ci sono i festeggiamenti ad Ostuni?

Assistente: Secondo giorno dei festeggiamenti in onore di Santa Maria della Stella presso la chiesa sita e lungo viale Oronzo 40 ad Ostuni fino al giorno di festa che sarà domenica 5 luglio.

Questa è l'interfaccia che si presenta quando l'utente carica un video YouTube e successivamente pone una domanda all'LLM con relativa risposta utilizzando la modalità di chat standard.

Videolezione Attiva

■ Radiostuni News del 27/06/2026

📖 TUTTA La Seconda Guerra Mondiale Dalla Prospettiva Tedesca | Documentario a PURO COLORE

■ Radiostuni News del 27/06/2026

Inizializza il Sistema (Inserisci url video)

Incolla l'URL di un video YouTube per scaricare la trascrizione e aggiungerlo all'archivio dell'applicazione.

Elabora ed
Archivia

✓ Video elaborato e indicizzato con successo.

Interroga l'LLM (Assistente)

Scegli lo stile di conversazione e fai una domanda. La chat è isolata e legata solo alla lezione attiva.

☒  Assistente Standard ☐  Dialogo Socratico

Chiedi
all'AI

Quando l'utente inserisce almeno due video, il menu a tendina consente di cambiare il video corrente;

Inizializza il Sistema (Inserisci url video)

Incolla l'URL di un video YouTube per scaricare la trascrizione e aggiungerlo all'archivio dell'applicazione.

**Elabora ed
Archivia**

✓ **Video elaborato e indicizzato con successo.**

Interroga l'LLM (Assistente)

Scegli lo stile di conversazione e fai una domanda. La chat è isolata e legata solo alla lezione attiva.

☒  **Assistente Standard** ☐  **Dialogo Socratico**

**Chiedi
all'AI**

Tu  **[Std]:** In che periodo Adolf Hitler fu nominato cancelliere?

Assistente:

Questa è l'interfaccia che si presenta all'utente mentre attende la risposta dell'LLM. Si vede lo skeleton di caricamento della risposta, che sparisce e verrà sostituito dal testo della risposta nel momento in cui l'LLM la fornisce.

3. Conclusioni

In questo progetto è stato realizzato un sistema RAG che consente il recupero della trascrizione di video YouTube e interrogazione tramite LLM dei contenuti estratti, tramite tecniche di tokenization, chunking semantico dinamico basato sulla similarità del coseno, retrieval dinamico basato su similarity threshold, utilizzo di database vettoriale FAISS, framework web Flask, LLM scaricati localmente e valutando i risultati ottenuti con metriche RAGAS. I risultati delle metriche si assestano su livelli discreti ma tendenti al basso, poiché questo tipo di LLM locali di taglia medio-piccola non sempre riescono a rispondere correttamente nonostante i chunk recuperati contengano la risposta esatta, mentre per quanto riguarda le domande dello stesso contesto ma di cui la risposta non è presente nella trascrizione e nelle domande fuori contesto, gli LLM hanno correttamente risposto di non sapere. Come sviluppi futuri si può pensare di utilizzare LLM più performanti oppure di utilizzare questo sistema per categorie di persone specifiche, ad esempio per studenti con disturbi dell'apprendimento.

REFERENCES

1. <https://arize.com/blog-course/evaluation-of-llm-rag-chunking-strategy/>
2. Lakshya Singh, Er. Alok Singh Jadaun, Prof. Brajesh Kumar Singh, 2026, A Cross-Lingual Retrieval-Augmented Generation Based Intelligent Teaching Assistant for Educational Video Content: Implementation and Evaluation, INTERNATIONAL JOURNAL OF ENGINEERING RESEARCH & TECHNOLOGY (IJERT) Volume 15, Issue 04 , April – 2026
3. Es, S., James, J., Anke, L. E., & Schockaert, S. (2024, March). Ragas: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th conference of the european chapter of the association for computational linguistics: system demonstrations* (pp. 150-158).